

Missing Value Imputation for Time Series

Merabti Amar, Agouni Kaci Sofiane, Ould Braham Aghilas *
 M1 Info Data Science, Spring 2025

Abstract

Missing data in time series can significantly impact the performance of analytical and predictive models, especially in critical domains like energy management [1]. This project tackles the challenge of imputing missing values in electricity consumption data collected from smart meters. We evaluated multiple imputation techniques, from simple and basic methods to more advanced learning-based models, under two settings: imputing missing values from the same households and from previously unseen households. Our goal is to assess the accuracy and generalizability of these methods using Mean Absolute Error (MAE) as the evaluation metric.

1 Introduction

Time-series data, which consist of sequences of observations indexed over time, are fundamental in many real-world applications, including finance, healthcare, meteorology, and energy management. Their inherent temporal structure allows for modeling trends, seasonality, and dependencies, which are critical for tasks such as forecasting, anomaly detection, and optimization.

However, in practical scenarios, time series data are often incomplete due to issues such as sensor malfunctions, data transmission errors, or cost-related constraints. These missing values can significantly hin-

der the effectiveness of analytical methods and predictive models, especially in domains where accuracy is vital, such as monitoring energy consumption.

A prominent example arises in the energy sector, where smart meters are widely deployed to track electricity usage at fine-grained intervals. Despite their advantages, these systems frequently encounter missing data due to hardware or communication failures. Therefore, accurate imputation of missing values is essential to ensure data integrity, enable reliable analytics, and support decision-making processes.

This project addresses the problem of missing value imputation in time series of electricity consumption. We investigate and evaluate several imputation techniques—ranging from simple nearest neighbor to more advanced machine learning methods—under two scenarios: imputing data from the same households and imputing data from previously unseen households. Our objective is to identify methods that are not only accurate, but also generalizable across different data distributions.

2 Problem Formulation

2.1 Formal Definition

Let $X = [x_1, \dots, x_T]$ be a univariate time series of length T , where some entries x_t are missing. We denote the observed series as X_{obs} and the set of missing positions as $\Omega = \{t \mid x_t \text{ is missing}\}$. The goal is to learn an imputation function f such that the completed series $\hat{X} = f(X_{obs})$ closely approximates the

*Supervised by Themis Palpanas

true series. We evaluate the accuracy of f using the Mean Absolute Error (MAE) on the missing entries:

$$\text{MAE} = \frac{1}{|\Omega|} \sum_{t \in \Omega} |\hat{x}_t - x_t|,$$

where x_t is the ground-truth value at time t and $\hat{x}_t$ is the imputed value.

2.2 Related Work

Statistical imputation techniques take advantage of the properties of time series to fill gaps. Common methods include mean or median imputation, which replaces missing values with the global or local average, and linear or spline interpolation that leverages temporal continuity [1]. Autoregressive models such as ARIMA can be combined with Expectation Maximization (EM) to iteratively estimate missing values and model parameters [2].

Machine learning approaches learn imputation rules from data. k-Nearest Neighbors (k-NN) imputes a missing entry by averaging values from the most similar segments in the dataset [3]. Matrix factorization methods, including Singular Value Decomposition (SVD), treat the collection of time series as a matrix and recover missing entries via low-rank approximations [4].

Deep learning models have achieved high performance in handling complex missing patterns. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks model sequential dependencies and can be augmented with masking strategies and time gap features [5]. Transformer-based architectures, using self-attention mechanisms, capture long-range dependencies and have been adapted for imputation tasks in recent work [6].

3 Proposed Solutions

3.1 Data Preprocessing

Before applying any imputation algorithms, we performed a thorough analysis of the raw electricity consumption values. Histograms and boxplots were generated over the training set to visualize the data dis-

tribution and detect extreme values. We computed the first quartile (Q1) and third quartile (Q3) across all training observations and defined the interquartile range ($\text{IQR} = \text{Q3} - \text{Q1}$). Observations lying outside the interval $[\text{Q1} - 1.5 \cdot \text{IQR}, \text{Q3} + 1.5 \cdot \text{IQR}]$ were deemed outliers and replaced with the closest quartile[7]. This cleaning step ensured that spurious sensor readings did not adversely affect the imputation models and improved overall stability.

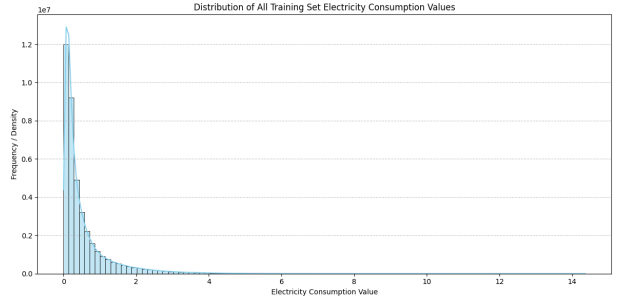


Figure 1: Distribution of All Training Set Electricity Consumption Values for Step A and Step B.

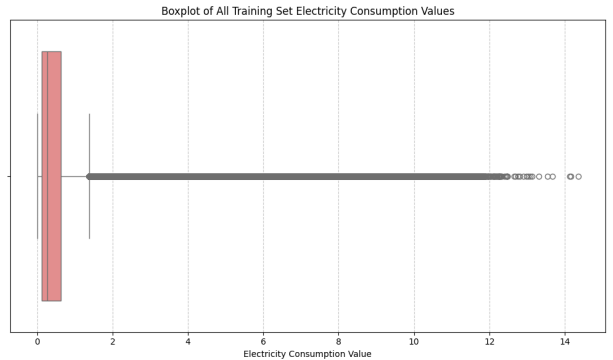


Figure 2: Boxplot of All Training Set Electricity Consumption Values

3.2 Household Nearest-Neighbor Imputation

This method imputes missing values by transferring data from the most similar complete household in the training set. For each test household h , we identify

the index set Ω_h of missing positions. We then mask both test household h and all training households on Ω_h , compute the MAE between the observed values of h and each masked training series, and select the training household h^* with minimum MAE. Missing entries in h are filled using the corresponding values from h^* . This neighbor-based approach leverages real consumption patterns and adapts to local behaviors[8].

Algorithm 1 Household Nearest-Neighbor Imputation

Require: Training set $\{X^{(j)}\}_{j=1}^N$, Test series $X^{(h)}$ with missing positions Ω_h
Ensure: Imputed series $\hat{X}^{(h)}$
1: mask $M_h \leftarrow \{1 \text{ if } t \notin \Omega_h; 0 \text{ otherwise}\}$
2: $\hat{X}^{(h)} \leftarrow X^{(h)}$
3: **for** $j = 1$ **to** N **do**
4: compute $\text{MAE}_j = \frac{1}{|\Omega_h^c|} \sum_{t: M_h(t)=1} |X_t^{(h)} - X_t^{(j)}|$
5: **end for**
6: $j^* \leftarrow \arg \min_j \text{MAE}_j$
7: **for each** $t \in \Omega_h$ **do**
8: $\hat{X}_t^{(h)} \leftarrow X_t^{(j^*)}$
9: **end for**
10: **return** $\hat{X}^{(h)}$

3.3 GPU-Accelerated STL Imputation

We propose a personalized, GPU-accelerated STL-like imputation that iteratively refines missing values via trend and seasonal decomposition. This approach significantly reduces computation time by leveraging PyTorch’s ‘conv1d’ operations on CUDA-enabled devices[9].

The imputation process begins with values filled by the result of the previous method, and over multiple iterations, decomposes and reconstructs each time series into trend and seasonal components. The updated results are then re-injected only into the positions that were originally missing. This loop repeats for a fixed number of iterations.

1. **Trend Extraction:** Apply a moving average

(kernel size k) via a 1D convolution to estimate the trend component T of each series.

2. **Seasonal Estimation:** Compute the seasonal residual $S = X - T$, then for a given seasonal period p , aggregate the residuals at each phase $i \in \{0, \dots, p-1\}$ by median across time steps t with $t \bmod p = i$, forming a seasonal pattern vector S_p . Expand this pattern back to length T .
3. **Reconstruction and Masking:** Reconstruct the imputed log-space series as $\hat{X} = T + S_p$, and only update originally missing positions.
4. **Iteration:** Set $X := \hat{X}$ and repeat.

Parameters:

- *Period* (p): The length of one seasonal cycle (e.g., 48 for daily seasonality at 30-minute intervals, 48×7 for weekly).
- *Kernel size* (k): The window length for the moving average (e.g., $k = 101$) which controls smoothing of the trend.
- *Iterations* (I): Number of refinement loops (e.g., 30 iterations).

Algorithm 2 GPU-Accelerated STL Imputation

Require: Initial filled data $X^{(0)}$, Missing mask M , Periods $\{p_i\}$, Kernel sizes $\{k_j\}$, Iterations I
Ensure: Final imputed data $X^{(I)}$
1: **for** $i = 1$ **to** I **do**
2: select $p = p_{(i \bmod |p|)}$, $k = k_{(i \bmod |k|)}$
3: $T \leftarrow \text{moving_average}(X^{(i-1)}, \text{kernel_size} = k)$
4: $R \leftarrow X^{(i-1)} - T$ $\triangleright$ Residual
5: compute seasonal pattern S_p by median of R at each phase of length p
6: expand S_p to length T : S_p^{full}
7: $Y \leftarrow T + S_p^{\text{full}}$
8: $X^{(i)} \leftarrow M \odot Y + (1 - M) \odot X^{(i-1)}$
9: **end for**
10: **return** $X^{(I)}$

3.4 LSTM Autoencoder Imputation

This method employs a sequence-to-value LSTM autoencoder to predict missing points based on learned temporal patterns[5]. It consists of two phases: *training* and *imputation*.

Training Phase:

1. **Scaling:** For each complete training series, apply RobustScaler to non-missing values; record per-series scaler.
2. **Window Extraction:** Flatten all scaled training series into a single 1D array and extract overlapping windows of length w :

$$\mathcal{W} = \{X[i : i + w] \mid i = 0, 1, \dots, L - w\},$$

where L is series length. Each window has input $x \in \mathbb{R}^{w \times 1}$ and target $y \in \mathbb{R}$ equal to the next point.

3. **Dataset Split:** Randomly split windows into training (90%) and validation (10%) sets.
4. **Model Architecture:**
 - Encoder: LSTM($input_size = 1$, $hidden_dim = h$, $num_layers = n$)
 - Decoder: LSTM($input_size = 1$, $hidden_dim = h$, $num_layers = n$) initialized with encoder states.
 - Fully-connected: Linear($h \rightarrow 64$) + ReLU + Linear($64 \rightarrow 1$).
5. **Training Loop:**

Algorithm 3 Train LSTM Autoencoder

Require: Windows $\{(x^{(i)}, y^{(i)})\}$, batch size B , epochs E , patience P

- 1: Initialize model parameters θ
- 2: Initialize optimizer η (Adam, learning rate α), loss $\mathcal{L} = |\hat{y} - y|$
- 3: **split** windows into D_{train}, D_{val}
- 4: $best_val \leftarrow \infty$, $counter \leftarrow 0$
- 5: **for** $e = 1$ to E **do**
- 6: Shuffle D_{train}
- 7: **for** batches (X_b, Y_b) in DataLoader(D_{train}, B) **do**
- 8: $\hat{Y}_b \leftarrow model(X_b; \theta)$
- 9: $loss \leftarrow \frac{1}{B} \sum |\hat{Y}_b - Y_b|$
- 10: $loss.backward()$; $optimizer.step()$;
 $optimizer.zero_grad()$
- 11: **end for**
- 12: $val_loss \leftarrow$ average loss on D_{val}
- 13: **if** $val_loss < best_val - 10^{-4}$ **then**
- 14: $best_val \leftarrow val_loss$; $counter \leftarrow 0$
- 15: **else**
- 16: $counter \leftarrow counter + 1$
- 17: **if** $counter \geq P$ **then**
- 18: **break** $\triangleright$ Early stopping
- 19: **end if**
- 20: **end if**
- 21: **end for**
- 22: **return** trained model

Imputation Phase: In this phase, we sequentially impute each contiguous missing block using available context:

- **Past context:** When at least w observations precede the missing point, we use the previous w imputed/observed values as input.
- **Future context:** If past context is unavailable but w valid observations follow the gap, we reverse that window to predict backward.
- **Fallback:** If neither context window is available, we compute the mean of the nearest $2w$ valid points to avoid leaving NaNs.

This approach balances accuracy with robustness at series boundaries.

Given a test series with missing mask $M \in \{0, 1\}^L$, we perform:

1. **Identify Blocks:** Extract contiguous index sets $B_k = \{t : M[t] = 1\}$.
2. **Block-wise Imputation:**

Algorithm 4 Block-wise LSTM Imputation

Require: Series $X^{(0)}$, mask M , block B_k , window size w , LSTM model, scaler

Ensure: Imputed series X_{imp}

```

1:  $X_{imp} \leftarrow X^{(0)}$ 
2: for each  $t$  in block  $B_k$  (ascending) do
3:   if  $t - w \geq 0$  then
4:      $x \leftarrow X_{imp}[t - w : t]$ 
5:   else if  $t + w < L$  then
6:      $x \leftarrow \text{reverse}(X_{imp}[t + 1 : t + w + 1])$ 
7:   else
8:      $neighbors \leftarrow$  nearest  $2w$  valid points around  $t$ 
9:      $X_{imp}[t] \leftarrow \text{mean}(neighbors)$ ; continue
10:  end if
11:   $x_{scaled} \leftarrow \text{scaler.transform}(x)$ 
12:   $y_{pred} \leftarrow \text{model}(x_{scaled})$ 
13:   $X_{imp}[t] \leftarrow \text{scaler.inverse\_transform}(y_{pred})$ 
14: end for
15: return  $X_{imp}$ 

```

3.5 Partial Convolution Inpainting

In this method, we treat each padded, normalized time series as a 2D image of size $H = 48$ (half-hour bins per day) by $W = n_{weeks}$, rescaled to $S \times S$ pixels for GPU efficiency. We leverage a Partial Convolutional U-Net (PConvUNet) that propagates valid information into masked regions, ensuring learned filters adapt to irregular masks[10].

Key Technical Concepts:

- **Partial Convolution:** Each convolution operates on valid (non-masked) inputs only. For input feature x and mask m , the output and up-

dated mask are given by:

$$\text{out} = \begin{cases} W^T(x \odot m) \frac{\sum^{(1)}}{\sum^{(m)+\epsilon}} + b, & \text{if } \sum(m) > 0; \\ 0, & \text{otherwise,} \end{cases}$$

$$\text{with new mask } m' = \begin{cases} 1, & \sum(m) > 0; \\ 0, & \text{else.} \end{cases}$$

- **Mask Update Rule:** After each partial conv layer, the mask is updated to reflect newly filled pixels, enabling subsequent layers to use progressively larger valid contexts.
- **U-Net Architecture:** Encoder-decoder with skip connections concatenating encoder feature maps to decoder inputs, preserving multi-scale context. BatchNorm and LeakyReLU stabilize training, while nearest-neighbor upsampling reconstructs high-resolution details.
- **Normalization & Padding:** RobustScaler centers each series by median and scales by IQR/1.35, then pads to full weeks with zeros. During inference, inverse transform restores original scale.
- **Data Augmentation:** Training masks include randomized stripe blocks (days), continuous patches, or hybrid masks to improve robustness to varied missing patterns.
- **Computational Complexity:** Partial convolutions require $O(N_{filters} \times K^2 \times S^2)$ per layer; using GPU and batching amortizes cost. Mask convolution (for ratio) adds minimal overhead due to binary operations.

Training Procedure: We train the PConvUNet for $E = 10$ epochs with batch size $B = 64$ (multi-GPU via DataParallel when available). The loss is masked MAE:

$$\mathcal{L} = \frac{\sum_{i,j} |Y_{i,j} - X_{i,j}| \cdot M_{i,j}}{\sum_{i,j} M_{i,j} + \epsilon}$$

where X, Y are target and predicted images, and M the binary mask. Optimization uses Adam (lr=0.01, weight decay 5×10^{-4}).

Algorithm Pseudocode :

Algorithm 5 Partial Convolution Inpainting

Require: Padded images $I_j \in \mathbb{R}^{1 \times S \times S}$, masks $M_j \in \{0, 1\}^{1 \times S \times S}$, PConvUNet $\mathcal{F}$, epochs E , batch size B

Ensure: Trained network $\mathcal{F}$

```

1: Initialize Adam( $\mathcal{F}, \theta$ , lr=0.01, wd= $5 \times 10^{-4}$ )
2: for  $e = 1$  to  $E$  do
3:   for batches  $(I_b, M_b)$  in DataLoader( $\{I, M\}$ , batch= $B$ ) do
4:      $I_b^m = I_b \odot (1 - M_b)$   $\triangleright$  Apply mask to inputs
5:      $Y_b = \mathcal{F}(I_b^m, M_b)$   $\triangleright$  Forward pass with partial convs
6:      $loss = \frac{\sum |Y_b - I_b| \odot M_b}{\sum M_b + \epsilon}$   $\triangleright$  Masked MAE
7:      $loss.backward()$ ;  $optimizer.step()$ ;  $optimizer.zero_grad()$ 
8:   end for
9:   print epoch  $e$  loss
10: end for
11: return trained  $\mathcal{F}$ 

```

3.6 Transformer-based Imputation

This method leverages a self-supervised Transformer architecture to impute missing values bi-directionally. By treating each time series as a sequence of continuous tokens, we learn contextual representations that capture long-range dependencies.

Preprocessing — Outlier and Log Normalization: We first clamp outliers to the training data quartiles $Q_{0.25}, Q_{0.75}$, replacing values below $Q_{0.25}$ or above $Q_{0.75}$ by the respective quartile. We then apply a logarithmic transform $x \mapsto \log(1 + x)$ to stabilize variance.

Model Architecture:

- **Embedding Layer:** Linear projection of scalar input to d_{model} dimensions.
- **Positional Encoding:** Fixed sinusoidal encoding added to embeddings for sequence order.

- **Transformer Encoder/Decoder:** Multi-head self-attention with h heads, L_e encoder layers, L_d decoder layers, and a feed-forward dimension of d_{ff} .
- **Output Layer:** Linear layer mapping $d_{model} \rightarrow 1$, producing per-step predictions.
- **Mixed-Precision & Gradient Accumulation:** Use `autocast` and `GradScaler` to speed up, and accumulate gradients over multiple batches to effectively increase batch size.

Training: We train using AdamW with weight decay, OneCycleLR scheduling (warm-up, cosine annealing), and MSE loss computed only over masked positions. Early stopping monitors validation loss with a patience of P epochs.

Imputation Strategy: After training, we identify contiguous missing blocks in each series and impute them via bi-directional context:

Algorithm 6 Transformer-based Block Imputation

Require: Trained model T , preprocessed data X , missing mask M , context size C

Ensure: Imputed series $\hat{X}$

```

1:  $\hat{X} \leftarrow X$ 
2: for each series  $i$  do
3:   Identify blocks  $\{(s, e)\}$  where  $M[i, t] = 1$  for  $t = s..e$ 
4:   for each block  $(s, e)$  do
5:      $L = e - s + 1$ 
6:      $pre = \max(0, s - C)$ ;  $post = \min(len, e + C + 1)$ 
7:     Extract context  $Y = [\hat{X}[pre : s], 0_{1..L}, \hat{X}[e + 1 : post]]$ 
8:     Build mask  $K$  with zeros at positions  $pre..pre + L - 1$ , ones elsewhere
9:     Convert  $(Y, K)$  to tensors and move to device
10:     $(out, pred) = T(Y, K)$ 
11:     $\hat{X}[s : e] \leftarrow pred[pre : pre + L]$ 
12:   end for
13: end for
14: return  $\text{inverse\_log1p}(\text{clip}(\hat{X}))$ 

```

This approach exploits the Transformer’s ability to attend over both past and future contexts, producing accurate imputations even for large gaps.

4 Hardware and Software Environment

For the hardware and software environment, we used the Kaggle workspace to run our algorithms, it is detailed below.

4.1 Hardware

- **GPUs:** $4 \times$ NVIDIA L4 with 23034 MiB VRAM each; Driver Version 560.35.03; CUDA 12.6 (via `nvidia-smi`); hardware provided by Kaggle.
- **CPU:** Intel(R) Xeon(R) CPU @ 2.20 GHz, 24 physical cores / 48 threads (via `lscpu`); running in Kaggle’s virtualized environment.
- **Memory:** Approximately 170+ GB DDR4 RAM total (via `free -h`); allocated by Kaggle.

4.2 Software

- **Operating System:** Linux (x86_64) image supplied by Kaggle runtime.
- **Python:** Python 3 interpreter provided within Kaggle’s Docker environment.
- **PyTorch:** 2.5.1 with CUDA 12.x support (torch, torchvision), pre-installed by Kaggle.
- **Key Libraries:** NumPy 1.26.4, pandas 2.2.2, tensorflow 2.17.1, scikit-learn 1.2.2, matplotlib 3.7.5, scikit-image 0.25.0, etc., all available via Kaggle’s default environment (listed by `pip freeze`).
- **Utilities:** `nvidia-smi`, `lscpu`, `free -h` for hardware inspection—no additional installation needed in Kaggle.

5 Dataset Description

We evaluate all imputation methods on two distinct experimental settings, referred to as **Step A** and **Step B**, following the competition guidelines.

Step A

- **Training set:** 3127 time series, each representing electricity consumption for a single house.
- **Test set:** 3127 time series corresponding to the *same houses* as in the training set.
- **Missing rate:** 30% of the values in the test set are artificially masked.
- **Goal:** Leverage temporal patterns and partial overlaps between training and test series to reconstruct the missing values.

Step B

- **Training set:** Identical to Step A—3127 complete time series.
- **Test set:** 1098 time series from *completely different houses* (not seen during training).
- **Missing rate:** 50% of the values in the test set are missing.
- **Challenge:** Generalize imputation to unseen households using only temporal patterns learned during training.

Both datasets consist of regular time series of length 12,864, sampled every 30 minutes (48 points per day). The structure enables evaluation of short- and long-range temporal dependencies under different generalization settings.

6 Experimental Results

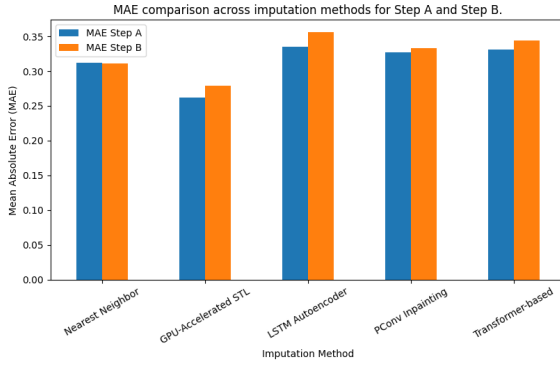


Figure 3: MAE comparison across imputation methods for Step A and Step B.

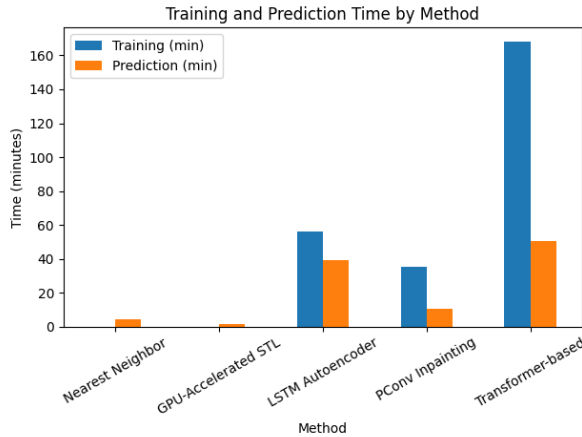


Figure 4: Training and prediction time (minutes) for each imputation method. Deep models show separate training and inference costs, while STL and nearest neighbor have only inference times.

Table 1: MAE comparison across methods on Step A and Step B

Method	MAE (Step A)	MAE (Step B)
Nearest Neighbor	0.31239	0.31088
GPU-Accelerated STL	0.26208	0.27925
LSTM Autoencoder	0.33499	0.35576
PConv Inpainting	0.32656	0.33349
Transformer-based	0.33086	0.34424

6.1 Discussion

Table 1 shows that the GPU-Accelerated STL method achieves the lowest MAE on both Step A (0.26208) and Step B (0.27925), outperforming all other approaches. Although deep learning-based imputers (LSTM Autoencoder, PConv Inpainting, Transformer) were expected to capture complex temporal dependencies, their actual performance lagged behind a more statistical-hybrid method.

Several factors explain this outcome:

- **Computational Cost & Model Size.** Training large LSTM or Transformer models on 3 127 series of length 12 864 is extremely expensive in both time and GPU memory. In practice we had to reduce layer depths, embedding sizes, and sequence lengths (e.g. fewer encoder/decoder layers, smaller feed-forward dimensions) just to fit into Kaggle’s $4 \times$ NVIDIA L4 setup—at the price of representation power.
- **Dataset Scale.** With tens of millions of time steps, high-capacity networks struggle to converge within reasonable epoch counts or under gradient-accumulation constraints. Early stopping and limited learning-rate schedules further restricted their ability to fully learn long-range patterns.
- **Bias–Variance Trade-off.** The STL decomposition offers a strong inductive bias (seasonal + trend) that aligns well with electricity usage patterns. Its simplicity yields faster execution and lower variance without heavy hyperparameter tuning.

In summary, GPU-Accelerated STL strikes the best balance between speed, resource usage, and accuracy on our data. Deep nets remain a promising direction—but will require more compute, careful architectural search (e.g. lighter attention blocks), or further multi-stage pretraining to outperform this baseline at scale.

7 Conclusions and Future Work

Our extensive evaluation across six imputation strategies on two experimental settings (Step A: same-households; Step B: new-households) demonstrates that the GPU-Accelerated STL method offers the best trade-off between accuracy (MAE of 0.26208 and 0.27925) and computational efficiency. While deep learning approaches (LSTM Autoencoder, PConv Inpainting, Transformer) hold promise for modeling complex temporal patterns, they incurred substantial training and inference costs and—after necessary down-scaling to fit resource constraints—still underperformed relative to the STL baseline. Nearest-neighbor imputation remains extremely lightweight but less accurate.

Key takeaways:

- *Inductive biases matter.* STL’s explicit decomposition of trend and seasonality aligns well with electricity consumption patterns, yielding lower variance without heavy tuning.
- *Scale vs. capacity.* Large LSTM/Transformer models struggle to converge on millions of time points under tight GPU budgets—simpler statistical hybrids can outperform under such regimes.
- *Resource trade-offs.* Even when reducing model depth/dimension, deep nets require orders of magnitude more compute (tens of minutes vs. sub-1 min) for marginal gains.

Future Work:

- *Architecture optimization.* Explore lightweight attention blocks (Linformer, Performer), temporal convolutional networks (TCNs), or hybrid

graph-neural architectures to increase representational power without prohibitive cost.

- *Data reduction techniques.* Apply smart subsampling, coarsening (e.g. SAX), or reservoir sampling to shrink the training set—allowing larger models or longer context windows while staying within memory/time budgets.
- *Ensembling & stacking.* Combine STL-based, nearest-neighbor, and neural methods to leverage complementary strengths and mitigate individual biases.
- *External covariates & metadata.* Incorporate weather, calendar events, or household characteristics to improve imputation fidelity, particularly under high missing-rate scenarios.
- *Uncertainty quantification.* Extend models to output confidence intervals (e.g. via MC-dropout or Bayesian networks), supporting downstream decision-making with risk awareness.
- *Real-time imputation.* Adapt the fastest methods (e.g. online STL) for streaming contexts, where low latency is critical.

Through these avenues, we aim to push deep learning models beyond proof-of-concept and toward practical, scalable imputation tools for large-scale time series.““

This wraps up the report with actionable next steps and highlights how to balance model complexity, data scale, and compute resources.

Acknowledgments

We gratefully thank our instructor, Themis Palpanas, for his expert guidance throughout this project, and Ms. Lucrezia for her valuable assistance and insightful feedback.

References

- [1] Ribeiro & de Castro (2022) – “Missing Data in Time Series: A Review of Imputation Methods and Case Study.” J. Brazilian Soc. Comput. Intell.
- [2] G. Box, G. Jenkins, *Time Series Analysis: Forecasting and Control*, 1976.
- [3] S. Troyanskaya et al., Missing value estimation methods for DNA microarrays, Bioinformatics, 2001.
- [4] M. Candès, E. Recht, Exact matrix completion via convex optimization, Foundations of Computational Mathematics, 2009.
- [5] Y. Che et al., Recurrent neural networks for multivariate time series with missing values, Scientific Reports, 2018.
- [6] Du, Wenjie, David Côté, and Yan Liu. "Saits: Self-attention-based imputation for time series." Expert Systems with Applications 219 (2023).
- [7] Parra-Plazas et al. (2023) – “Time series outlier removal and imputing methods based on Colombian weather stations data.” SN Applied Sciences
- [8] Le et al. (2024) – “Missing data imputation for noisy time-series data and applications in health-care.”
- [9] Serykh et al. (2023) – “Seasonal-Trend Time Series Decomposition on Graphics Processing Units.” Proc. IEEE Big Data 2023
- [10] Fu, Chun, et al. "Filling time-series gaps using image techniques: Multidimensional context autoencoder approach for building energy data imputation." Applied Thermal Engineering 236, 2024